

머신러닝 개념 + PyTorch 대응 정리

로봇 21기 예비인턴 최윤서

1. Machine Learning

1.1 Machine Learning이란?

- 훈련된 **Data set**을 활용하여 **Data**의 **pattern &** 관계를 파악하는 것
- 새로운 데이터를 예측하거나 의사 결정에 사용함

1.2 Machine Learning의 종류

- 1) 지도학습
 - 정답을 미리 알려줌
 - 라벨링된 훈련 데이터를 사용해서 **input & output**의 관계를 학습함
 - 예: YOLO 공, 골대 검출
- 2) 비지도학습
 - 라벨링되지 않은 데이터를 활용
 - 가이드 없이 **pattern & relationship**을 파악함
 - 예: 필드 색상을 **k-means**로 묶기
- 3) 강화학습
 - 모델의 행동 **or** 결과에 따라 **Reward**와 **Penalty**를 줌
 - 시행착오를 통해 학습함
 - 보상 극대화 과정에서 의사 결정 패턴을 개선함
 - 예: 휴머노이드 보행

1.3 Machine Learning 작동 과정

- 1) 전처리 단계
 - 데이터 클렌징, 변환, 필터링 과정을 거쳐서 **Training Dataset** 생성
 - **Training Data**의 완성도 = 모델의 효율성과 성능을 좌우함
- 2) 모델링 단계
 - 알고리즘 선택 -> 훈련 -> 테스트 => 가장 성능이 좋은 모델을 생성함
 - 알고리즘 선택: **pattern**을 찾을 알고리즘을 선택
 - 훈련: 1단계에서 생성된 **Training Dataset**으로 모델의 최적 계수 찾아냄
 - 테스트: 훈련용과 별개의 **Dataset**으로 모델의 성능을 평가
 - 테스트 과정 후에 다시 알고리즘 선택 단계로 넘어가서 더 좋은 성능의 모델이 있는지 계속 반복하며 찾아봄
- 3) 예측 단계
 - 실제 **Data** -> 모델에 적용 -> 결과 예측
 - 맘에 드는 모델을 실무에서 직접 사용
 - 데이터 투입 방식: 일괄 처리 **or** 스트리밍(단계별 처리 방법)

2. Perceptron

2.1 퍼셉트론이란?

- 인공 신경망으로 다수의 입력을 주면 하나의 결과를 내보내는 알고리즘

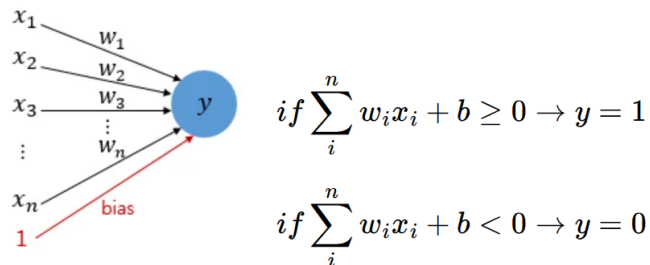
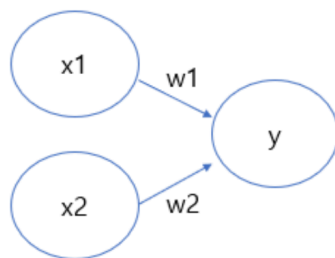


그림1

그림2

- x : 입력값, w : 가중치, y : 출력값
- 그림 2와 같이 입력값과 가중치의 곱을 모두 더한 값에 편향(bias)를 더한 값이 0 이상이나, 미만이나에 따라서 출력값이 0과 1로 결정됨
- 이렇게 뉴런에서 출력값을 변경시키는 함수를 활성화 함수라고함.
즉, 그림2가 활성화 함수가 작동한 사진(3번에서 더 자세히 설명)

2.2 단층 퍼셉트론



단층 퍼셉트론 도식

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

- 퍼셉트론 중에 가중치를 갖는 층이 한층이라면 단층 퍼셉트론이라고함
- 단층 퍼셉트로 수식은 오른쪽 그림과 같이 계산됨

2.3 pytorch 퍼셉트론 구현

1) 대응 관계

개념	pytorch
입력 개수	in_features
출력 개수	out_features
가중치	weight(shape: out_features, in_features)
편향	bias(shape: out_features)
가중합 계산	nn.Linear의 forward

2) pytorch 구현

```
import torch
import torch.nn as nn

def make_gate(w, b):
    layer = nn.Linear(2, 1)
    with torch.no_grad():
        layer.weight.copy_(torch.tensor([w]))
        layer.bias.copy_(torch.tensor([b]))
    return layer

def step(x):
    return (x > 0).float()

AND = make_gate([ 0.5, 0.5], -0.7)
NAND = make_gate([-0.5, -0.5], 0.7)
OR = make_gate([ 0.5, 0.5], -0.2)

inputs = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])

for name, gate in [("AND", AND), ("NAND", NAND), ("OR", OR)]:
    print(name, step(gate(inputs)).flatten().tolist())
```

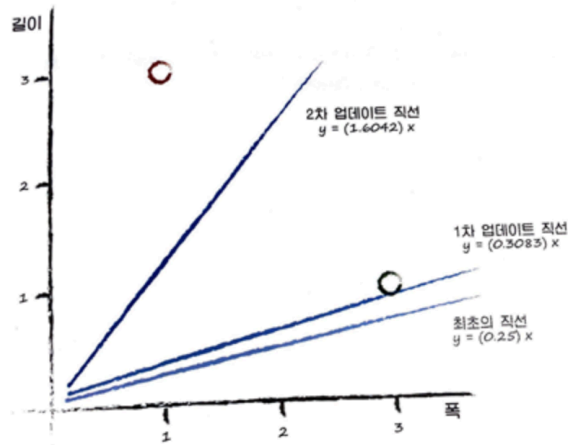
nn.Linear(2,1): 입력 x1, x2를 받아 하나의 값을 내는 구조
weight의 shape이 (1,2) 이므로 torch.tensor([w]) 처럼 대괄호 하나 더 씌워주기
step함수는 2.2의 수식을 그대로 구현한 것
torch.no_grad()는 이 구간의 연산을 기울기 계산에 포함하지 않겠다는 의미

2.4 선형적 분류

다층 퍼셉트론으로 넘어가기 전 왜 다층 퍼셉트론이 필요한가?에 대해 먼저 설명하고 넘어가겠다.

선형 함수는 말 그대로 $y = ax + b$ 의 형태로 입력값을 받아 출력값을 모든 x값에 대해 출력했을 때 그 형태가 직선으로 나오게 된다.

a와 b값을 바꾸면 직선의 기울기와 위치를 변화시킬 수 있는데 이때 선형 분류자를 학습시켜서 기울기와 위치를 잘 조절해서 데이터 값을 잘 분류하도록한다.



학습 과정은 임의의 a, b 로 시작하여 오차를 통해 기울기를 변화시킨다.
그러나 직선이 오차만을 기준으로 업데이트하면 최종적으로는 그저 마지막 학습데이터에 맞춰진 업데이트 결과만 얻게 된다.

$$y = Ax, t = (A + \Delta A)x, E = t - y \rightarrow E = \Delta A, \text{방정식: } \Delta A = E/x$$

오차에 기초해 매개변수 A 업데이트

이를 해결하기 위해 업데이트 정도를 조금씩 조절할 수 있는 ‘학습률’이라는 개념이 도입된다. 각각의 새로운 a 로 바로 점프하면서 수정하는 것이 아니라 일부분만 조금씩 업데이트 하는 것이다.

$$\rightarrow \Delta A = L(E/x)$$

L은 학습률

즉, 기존에 여러번 학습에 기초해 업데이트된 값을 유지하면서, 학습 데이터가 제시하는 방향으로 조금씩만 움직이는 것이다.

그러나 이런 학습률에도 문제점이 있다. 학습 데이터 자체에 문제가 있는 경우, 학습 데이터 자체가 오차 또는 노이즈를 가지는 경우 등은 오히려 업데이트 정도를 조정하면서 학습시키면 오차나 노이즈를 극대화 시킬 수 있다.

또한, 선형 분류자의 한계점은 직선 영역만 표현할 수 있다는 것이다. 데이터들의 차이가 확실히 구분되지 않는다면 두가지 종류 조차도 분류하지 못할 가능성이 크다.
따라서 다층 퍼셉트론이 나오게 되었다.

2.5 다층 퍼셉트론

단층 퍼셉트론은 입력층과 출력층만 존재하지만 다층 퍼셉트론은 중간에 은닉층이라고 불리는 층이 추가된다.

층을 여러겹으로 쌓아가면서 선형 분류만으로 풀지 못했던 문제를 비선형적으로 풀 수 있게 된다. 각 층에서 오차는 역전파 알고리즘을 통해 업데이트해나간다.
역전파 알고리즘은 6번에서 더 자세히 설명하겠다.

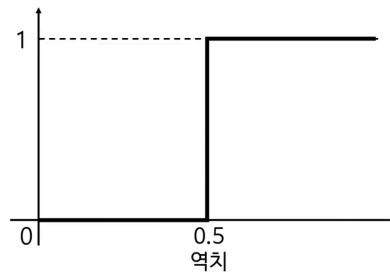
3. 활성화 함수(Activation Function)

3.1 활성화 함수란?

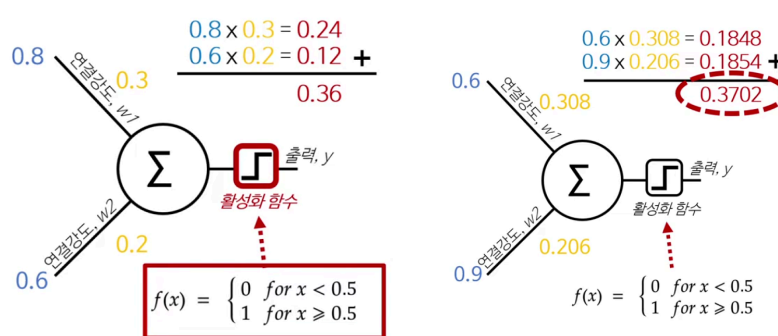
2.1의 그림 2와 같이 input신호를 비선형적인 값인 0과 1로 변환하여 선형회귀, 로지스틱 회귀 등으로 해결되지 않는 복잡한 문제들을 해결할 수 있게 돕는 함수

3.2 활성화 함수 종류

1) 계단 함수



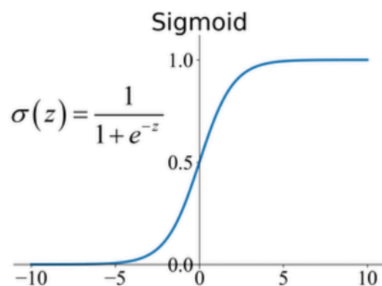
- 함수의 값이 역치 이하일 경우 -> 0 출력
- 함수의 값이 역치 이상일 경우 -> 1 출력



- 가중치가 업데이트 된다고하여도 출력값을 0이되고 극단적인 경우 0.001 인 경우와 4.999 인 경우 모두 0을 출력함
- 분명히 4.999가 더 예측을 잘 한것인데 이를 묵살시킴

2) sigmoid 함수

계단함수의 단점을 보완하기위해 학습의 효율성을 내부값의 차이를 오차에 반영하도록 수정



시그모이드함수는 다층 신경망의 혁신을 시작한 결정적인 계기인 활성화 함수이다. 왜 시그모이드함수가 다층 신경망의 혁신을 가져왔는가?

다층 퍼셉트론이 학습할 때 사용하는 오류역전파는 경사하강법을 사용한다.
이때 경사하강법을 사용하려면 활성화함수가 미분 가능한 함수여야하기 때문에
시그모이드함수가 다층 퍼셉트론의 학습 방식의 발전에 큰 영향을 미친 것이다.

시그모이드함수가 가지는 장점은 계단함수로 표현할 수 없었던 내부값의 차이를 표현할
수 있기 때문이다.

즉, 계단함수처럼 0 또는 1이 아니라 내부값에 따라 0과 1 사이의 값을 출력할 수 있다.

출력값이 0과 1 사이이기에 0 또는 1 보다는 훨씬 다양한 오차를 계산할 수 있다.

또한, 0~1사이이기 때문에 백분을 확률로 표현할 수도 있다.

즉, 신경망의 출력값이 단순한 숫자가 아니라 의미를 가진 숫자로 출력되는 것에서
시그모이드함수는 큰 의미가 있다.

그러나 시그모이드 함수에도 단점이 존재한다.

기울기 소실 문제와 학습 과정 중 나타나는 지그재그 현상이 대표적이다.

시그모이드함수의 양 끝쪽 기울기는 0에 가깝다.

이는 입력값의 차이가 커도 출력값의 차이는 점점 미미해진다는 것을 의미한다.

시그모이드 출력값은 항상 양수이기 때문에 가중치의 값 또한 0과 1 사이의 양수라고
가정한다면 상위 노드의 입력값은 양수가 되고 상위 계층으로 올라갈수록 입력값이
점점 커지게 된다.

입력값이 점점 커진다면 기울기는 점점 0에 가까워진다.

기울기가 0이 되면 오류역전파의 학습 효과가 0에 가까워진다.

즉, 기울기소실이 발생한다.

또한, 두번째 문제는 시그모이드의 출력값이 항상 양수라는 것이다.

입력값이 항상 양수가되고 출력값의 오차가 음수가 된다면 가중치는 음수가되고

입력값이 양수이고 출력값의 오차가 양수라면 가중치는 양수가 된다.

따라서, w_1 , w_2 공간에서 움직일 수 있는 공간을 시각적으로 표현해보면 아래 그림5과
같이 표현할 수 있다.

학습 과정을 통해 최적의 가중치값으로 가고싶다고 했을 때

지그재그의 형태로 최적의 값으로 이동하게되어 비효율적으로 움직인다는 단점이 있다.

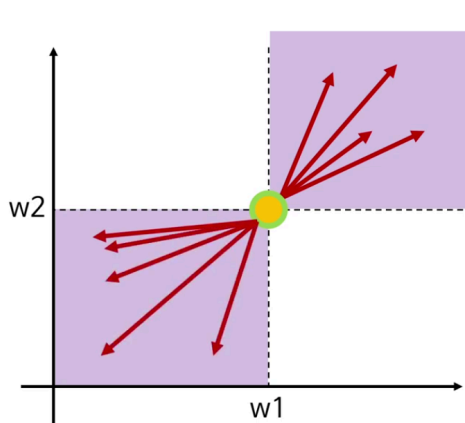


그림5

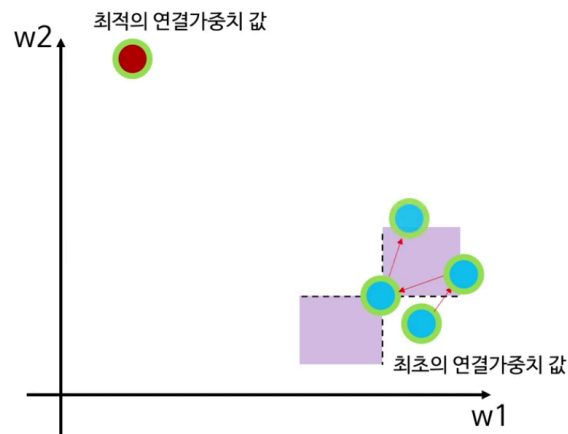
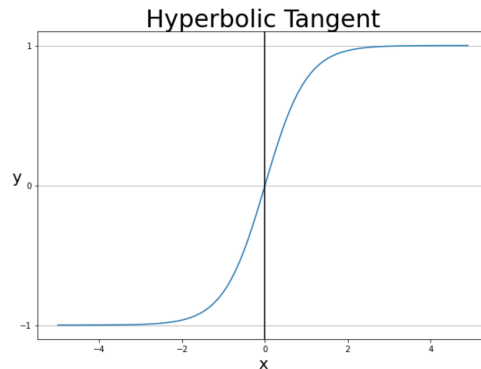


그림6

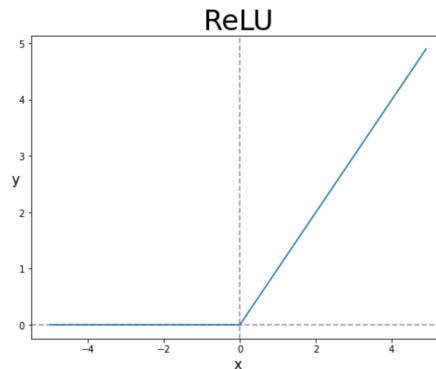
3) tanh 함수

따라서 시그모이드 함수와 비슷하게 생겼지만 출력값이 $-1 \sim 1$ 인 tanh 함수가 나왔다.



4) ReLU

tanh 함수도 해결할 수 없었던 기울기 소실 문제를 해결할 수 있는 ReLU 함수가 생겼다.



입력값이 양수일 때는 기울기가 늘 1로 고정되어있어 기울기 소실이 발생할 여지가 없다. 따라서 신경망의 깊은 층까지 오류역전파를 통한 학습이 가능해졌다.

동시에 렐루 함수 공식은 입력값이 음수면 0, 양수면 자기 자신을 반환하는 단순한 공식이다보니 경사하강시 다른 활성화 함수에 비해 학습속도가 매우 빠르다는 장점이 있다.

하지만 렐루 함수도 단점이 존재한다.

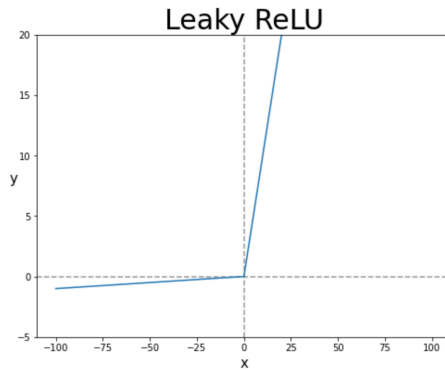
입력값이 음수일 경우에는 기울기가 0이 되어 모든 값을 0으로 반환하다보니 가중치가 업데이트되지 않는 현상이 발생할 수 있다.

즉, 가중치가 업데이트되는 과정에서 가중치합이 음수가 되는 순간 렐루는 0을 반환하기 때문에 해당 뉴런은 그 후로 0만 반환하고 아무것도 변하지 않는 현상이 발생할 수 있다.

이렇게 Dead Neuron을 초래하는 현상을 Dying ReLU 현상이라고한다.

5) Leaky ReLU

Dying ReLU 현상을 해결하기 위해 Leaky ReLU 함수가 생겼다,

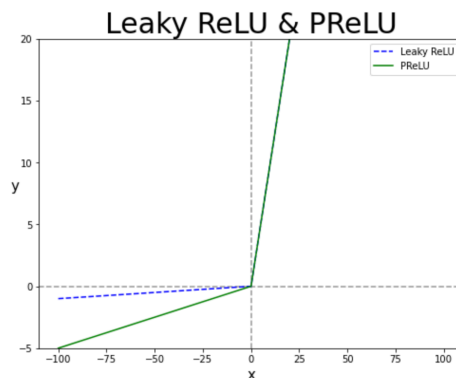


음수 입력값에 작은 기울기를 주어서 신경망 노드들의 입력값이 음수일 때도 학습이 가능하도록 수정한 함수이다.

그러나, 기울기를 수정하여 음수 입력값에서 사용한다면 음수 입력값에서 선형성이 생기게 되고 그로 인해 복잡한 분류에서는 사용할 수 없다는 한계가 생긴다. 만약, 음수 입력값이 아주 중요한 상황의 학습이라면 제한적으로 사용하는 경우는 있지만 보편적으로는 잘 사용하지 않는다.

6) PReLU

PReLU 함수도 Leaky ReLU 함수와 동일하게 Dying ReLU 현상을 막기 위해 생겨났다.

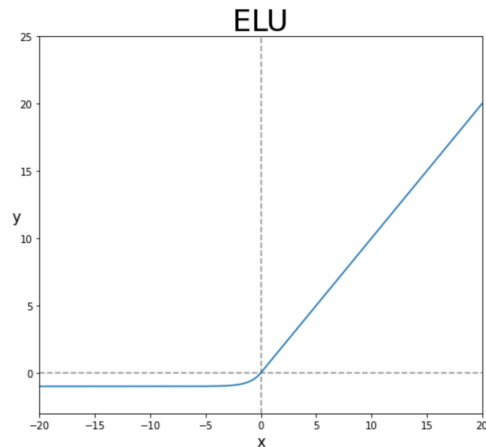


Leaky ReLU와 성격이 상당히 비슷하지만 음수의 계수인 알파를 가중치 매개변수처럼 학습되도록 역전파에 알파값이 변경되었기 때문에 대규모 이미지 데이터셋에서는 ReLU보다는 좋다는 평가도 있지만 소규모 데이터 셋에서는 적절치않을수도있다.

또한 PReLU도 선형성을 띄기 때문에 복잡한 분류에서 사용하지 못할 수 있으므로 주의해서 사용해야한다.

7) ELU

ReLU와 유사하게 생겼으면서도 $\exp$ 를 이용해 그래프를 부드럽게 만들고 미분 시 0에서 끊어지는 ReLU와 달리 미분해도 부드럽게 이어지도록 수정한 함수



ELU는 ReLU의 대표적인 대안 방법 중 하나이다.
ReLU와 달리 입력값이 음수인 경우에도 값을 생성해낼 수 있고
Leaky ReLU와 PReLU와 달리 음에서도 비선형적이기 때문에
복잡한 분류에서도 사용 가능하다.

일반적으로 알파는 1로 설정하며 이 경우 $x = 0$ 에서 급격하게 변하지 않고 모든 구간에서
매끄럽게 변하므로, 경사하강법에서 수렴 속도가 매우 빠르다.

그러나, ReLU에 비해 성능이 크게 증가하지 않는다는 연구 결과가 있으며 $\exp$ 때문에
연산량이 늘어났기에 학습속도가 느려 잘 사용하지 않는 추세이다.

3.3 PyTorch로 Activation Function 구현하기

1) sigmoid ([공식 문서](#))

```
import torch
import torch.nn as nn

class DenseBlock(nn.Module):
    def __init__(self, in_dim, out_dim):
        super(DenseBlock, self).__init__()
        self.dense = nn.Linear(in_dim, out_dim)
        self.act = nn.Sigmoid() # activation function

    def forward(self, x):
        out = self.act(self.dense(x))
        return out
```

2) tanh(Hyperbolic tangent) ([공식 문서](#))

```

import torch
import torch.nn as nn

class DenseBlock(nn.Module):
    def __init__(self, in_dim, out_dim):
        super(DenseBlock, self).__init__()
        self.dense = nn.Linear(in_dim, out_dim)
        self.act = nn.Tanh() # activation function

    def forward(self, x):
        out = self.act(self.dense(x))
        return out

```

3) ReLU([공식 문서](#))

```

import torch
import torch.nn as nn

class DenseBlock(nn.Module):
    def __init__(self, in_dim, out_dim):
        super(DenseBlock, self).__init__()
        self.dense = nn.Linear(in_dim, out_dim)
        self.act = nn.ReLU() # activation function

    def forward(self, x):
        out = self.act(self.dense(x))
        return out

```

4) Leaky ReLU ([공식 문서](#))

```

import torch
import torch.nn as nn

class DenseBlock(nn.Module):
    def __init__(self, in_dim, out_dim):
        super(DenseBlock, self).__init__()
        self.dense = nn.Linear(in_dim, out_dim)
        self.act = nn.LeakyReLU() # activation function

    def forward(self, x):
        out = self.act(self.dense(x))
        return out

```

5) ELU ([공식 문서](#))

```
import torch
import torch.nn as nn

class DenseBlock(nn.Module):
    def __init__(self, in_dim, out_dim):
        super(DenseBlock, self).__init__()
        self.dense = nn.Linear(in_dim, out_dim)
        self.act = nn.ELU() # activation function

    def forward(self, x):
        out = self.act(self.dense(x))
        return out
```

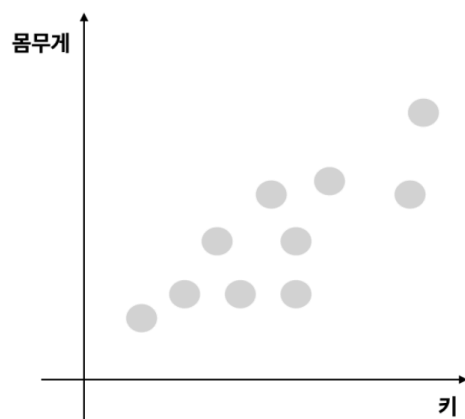
4. 비용함수(Cost Function)

4.1 비용함수란?

내가 만든 모델이 실제 정답과 얼마나 다른지 측정하는 수단이다.
다시 말해, 실제 값과 모델이 예측한 값 간의 차이를 의미한다.

비용 함수는 값이 클수록 해당 모델의 정확도가 낮다는 것을 의미한다.

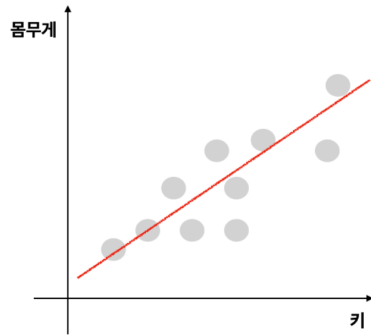
4.2 작동 과정



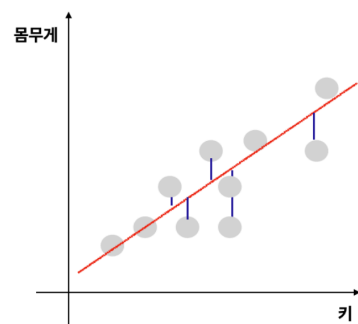
실제 데이터의 분포

위와 같은 데이터 분포가 있다고 가정해보자. 일반적으로는 키가 커짐에 따라 몸무게도 어느정도 증가하는 추세이다.

이때 이 데이터들을 가장 잘 설명하고 잘 표현할 수 있는 하나의 직선을 그린다면 어떻게 그릴 수 있을까?



위와 같이 대충 생각해보면 이런 식으로 그릴 수 있을 것이다.
 이 선을 어떻게 어떤 방식으로 그려야 그것을 최적의 선이라고 할 수 있는가?
 이 때 판단 기준이 되는 것이 바로 오차이다.



위 그림과 같이 파란색 선이 오차이고 파란색 선의 총합이 클수록 오차가 크다는 것을 의미한다.

이제는 모델의 오류를 줄이기 위해 현재 모델이 어느 정도 오류를 뱉는지 측정하고 이를 줄여나가는 과정이 필요하다.

오차를 단순히 더해버리면 +, -값인 오차들이 함께 전재하므로 정확한 오차합을 계산할 수 없다.

따라서 각 오차를 제곱한 뒤 그것을 합산하는 방식 **or** 각 오차에 절댓값을 씌운 뒤 합산하는 방법을 사용한다.

전자의 경우를 **MSE**, 후자의 경우를 **MAE**라고 한다.

MSE는 오차를 제곱하기 때문에 큰 오차에 대해 더 큰 패널티를 부과하여 큰 오차를 피하도록 만든다. 그러나 데이터의 제곱 단위이므로 해석이 다소 어렵다는 특징이 있다.

MAE는 예측 값과 실제 값 사이의 평균 절대 차이를 제공하여 직관적으로 해석하기 쉽고 **MSE**에 비해 절대값을 사용하기 때문에 이상치에 덜 민감하다는 특징이 있다.

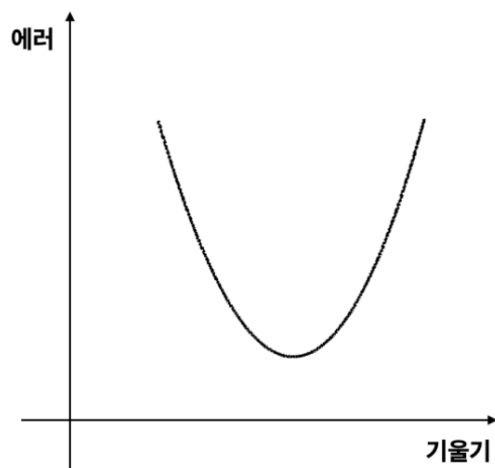
$$MAE = \frac{1}{n} \sum_i^n |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

보편적으로는 전자의 계산 방법을 사용한다.

각 오류를 제곱한 뒤 더하는 방식을 통해 오류값들을 아래 사진과 같이 하나의 수식으로 표현할 수 있다.

결론적으로 이 포물선 형태의 함수를 비용함수라고 한다.



여기서 오차를 최소화하려면 포물선의 최저점을 찾아야하고 최저점을 찾는 과정에서 5번에 설명할 경사하강법이 사용된다.

경사하강법에 대한 자세한 내용은 5번에서 설명하겠다.

4.3 pytorch로 비용함수 구현하기

1) 대응 관계

개념	pytorch	용도
MSE	nn.MSELoss()	회귀
MAE	nn.L1Loss()	회귀, 이상치에 덜 민감
이진 분류	nn.BCELoss(): 입력이 확률 nn.BCEWithLogitsLoss(): 입력이 logit	시그모이드 함수 출력

2) 코드 구현

```
pred = torch.tensor([2.5, 0.0, 2.1])
target = torch.tensor([3.0, -0.5, 2.0])

mse = nn.MSELoss()(pred, target)
mae = nn.L1Loss()(pred, target)
print(mse.item(), mae.item())

print(((pred - target) ** 2).mean().item())
print((pred - target).abs().mean().item())
```

`nn.MSELoss`는 4.2의 MSE 수식 $MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$ 을 계산

`nn.L1Loss`는 MAE 수식 $MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$ 을 그대로 계산함

호출 순서는 `loss_fn(예측값, 정답)` 이다.

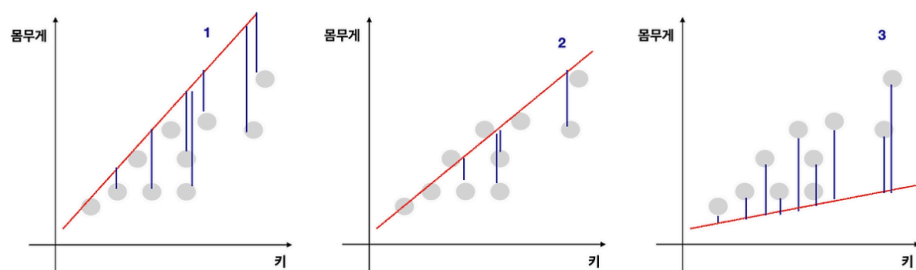
기본 값이 `reduction = 'mean'` 이라서 문서의 $1/n$ 평균과 같은 결과가 나온다.
(`sum`, `none` 도 선택 가능)

맨 아래 두 줄은 직접 수식으로 계산한 값을 확인하기 위한 용도임

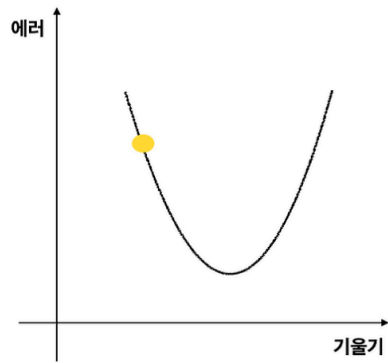
5. 경사하강법

5.1 작동 과정

경사하강법이란 주어진 비용 함수에서 모델의 파라미터의 최적 값을 찾는 머신러닝과 딥러닝의 최적화 알고리즘 중 하나이다.



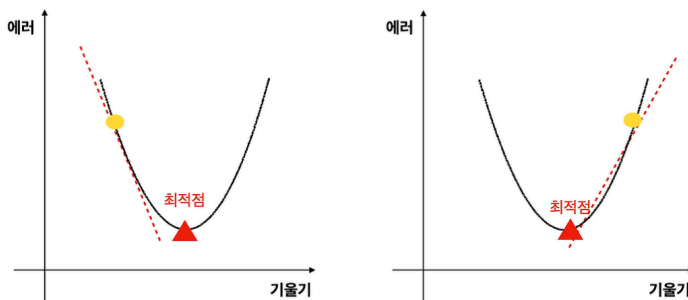
위 그림과 같이 기울기를 조금씩 변경해보면 기울기마다 오차가 달라짐을 알 수 있다.
4번에서 봤던 포물선 그래프와 같이 기울기에 따라 오차값을 표현하면
아래 그래프와 같이 표현할 수 있다. ($n = 1$ 인 경우)



이제부터 포물선의 최저점을 찾는 경사하강법의 원리에 대해 설명하겠다.

- 1) 임의의 한 점을 찍고 그 점에서 미분해서 기울기를 구한다
- 2) 그 점에서 접선의 기울기를 바탕으로 파라미터를 어느 방향으로 얼마만큼 움직일지 결정한다.
- 3) 기울기가 0이 되는 지점까지 반복해서 이동한다.

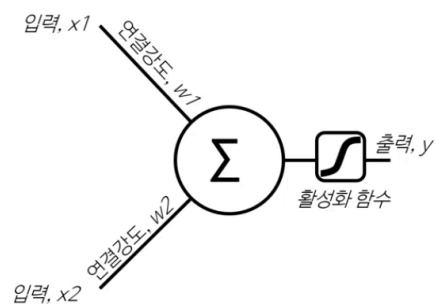
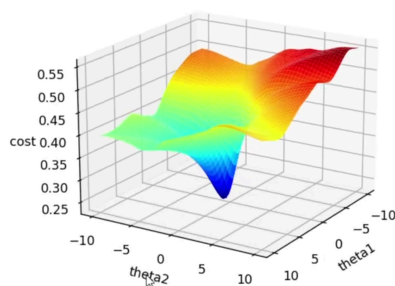
2)번 과정에서 만약 기울기가 음수라면 파라미터를 오른쪽으로 이동하고 기울기가 양수라면 왼쪽으로 이동하게 된다.



그런데, 비용함수는 그저 2차함수인데 그냥 근의 공식으로 구하거나 방정식을 미분해서 0이 되는 구간을 구하면 되는거 아닌가? 라는 생각이 들 수 있다.

그러나 대부분의 경우 딥러닝의 비용함수는 굉장히 복잡하다.

아래 사진과 같이 입력이 두개이고 시그모이드함수를 활성화함수로하는 퍼셉트론의 비용함수만 보더라도 아래 그림과 같이 3차원의 굉장히 복잡한 비용함수가 생성되기 때문에 경사하강법이 필요하다고할 수 있다.



5.2 pytorch로 경사하강법 구현하기

1) 대응 관계

개념	pytorch
파라미터 theta	model.parameters() 또는 requires_grad=True인 텐서
비용 함수	loss
기울기	param.grad
학습률	lr
업데이트	optimizer.step() / torch.optim.SGD

2) 코드 구현

```
w = torch.tensor(5.0, requires_grad=True)
optimizer = torch.optim.SGD([w], lr=0.1)
```

```
for i in range(50):
    loss = (w - 2) ** 2
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if i < 3:
        print(i, w.item(), w.grad.item())
print(w.item())
```

“임의의 점에서 시작 → 미분으로 기울기 계산 → 기울기 반대 방향으로 이동 → 반복”의 과정이 `zero_grad()` 부터 `step` 까지 `for` 문 안에서 반복됨

`w = torch.tensor(5, 0)`은 임의의 한 점

`loss.backward()`는 미분해서 기울기 구하기

`optimizer.step()`은 이동 단계

`lr`을 키우면 큰 폭으로 이동하고 너무 크면 최저점을 지나버릴 수 있음

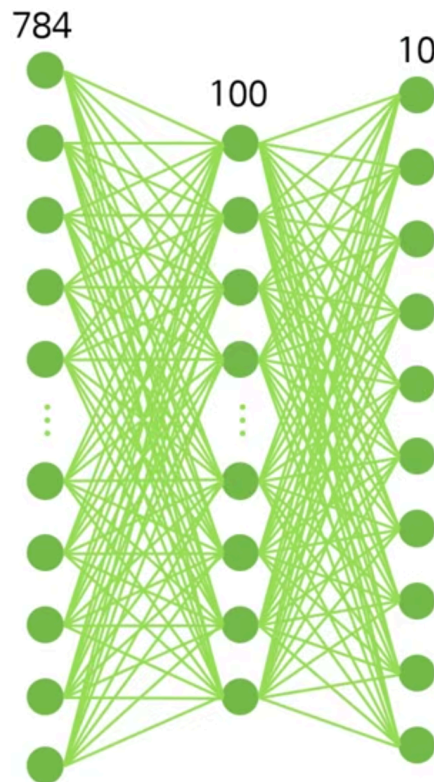
6. 역전파

6.1 배경

2번에서 설명한 바와 같이 역전파는 다층 퍼셉트론을 풀이하는 데에 큰 기여를 한 알고리즘이다.

1960년대부터 80년대 초에는 단일 퍼셉트론과 같은 얇은 신경망에 초점이 맞추어져있었고 다층 퍼셉트론은 역전파 알고리즘이 생기기 전에는 풀기 매우 어려운 문제였습니다.

만약 역전파 알고리즘을 사용하지않고 다층 퍼셉트론을 계산해보자



입력층: 784, 은닉층: 100, 출력층: 10인 단순한 3층 신경망을 가정하고 생각해보자.
연결가중치의 개수는 $784 \times 100 + 100 \times 10 = 79,400$
편향 개수는 $100 + 10 = 110$
한개의 이미지 손실 계산은 79,400번의 연산 필요

이때, 한 개의 파라미터를 업데이트(60,000개를 학습데이터로 사용)하기 위해서는
 $79,400 \times 60,000 = 4,764,000,000$ 이고

모든 파라미터들에 대해해 각각 연산해야하므로
 $4,764,000,000 \times 79,511 = 378,790,404,000,000$ 이다.

이를 시간으로 환산하면 약 123시간이 걸리게 된다.

3층으로만 만들어도 엄청나게 오래 걸리는데 100층을 넘어간다면 상상도 할 수 없는 시간만큼 학습시켜야했을 것이다.

이러한 문제점을 해결해준 알고리즘이 역전파 알고리즘이다.

6.2 작동 과정

1) 순전파

만약 입력값이 주어진다면 가중치와의 곱을 합하여 은닉층 노드에 넣는다
그 다음 은닉층 노드의 활성화 함수를 거쳐 **output**을 계산한다.

활성화 함수를 거쳐 나온 **output**을 다음 가중치와 곱해 새로운 값을 만들고 이를 다시
활성화 함수에 넣어 출력층에 입력한다.

2) 손실 계산

만약 손실 함수를 **MSE**를 사용한다면 아래 사진과 같은 공식을 사용할 수 있다.

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

$n = 1$ 이라고 가정하고 $L = (y - \text{출력값})^2$ 이다.

보통 $y = 1$ 이라고 가정(이상적인 값)하고 계산하므로 1에서 출력값을 뺀 값의 제곱이
결국 최종 오차가 된다.

3) 역전파

2번 과정에나온 오차를 활용하여 다시 반대 방향으로 진행하면서 가중치의 값을
업데이트 하는 과정이다.

new 가중치 = 현재 가중치 + $(-\partial L / \partial w_2) \times \text{학습률}$ 이다.

이때 학습률은 새로운 가중치에 어느 정도의 영향을 미치면서 가중치의 값을 수정할
것인지를 나타내는 값이다.

6.3 pytorch로 구현하기

1) 대응 관계

개념	pytorch
기울기를 구할 가중치	<code>requires_grad = True</code> 텐서 (<code>nn.Module</code> 의 파라미터는 자동)
순전파	일반 연산 (<code>model(x)</code>)
연쇄 법칙으로 기울기 계산	<code>loss.backward()</code>
계산된 기울기	<code>w.grad</code>
기울기 추적 중단	<code>torch.no_grad()</code> , <code>.detach()</code>

2) 코드 구현

```
x = torch.tensor(2.0)
y = torch.tensor(1.0)
w = torch.tensor(0.5, requires_grad=True)
```

```
y_hat = torch.sigmoid(w * x)
loss = (y - y_hat) ** 2
loss.backward()
print(w.grad)
```

```
s = torch.sigmoid(w * x)
print(2 * (s - y) * s * (1 - s) * x)
```

`loss.backward()`가 연쇄법칙을 자동으로 수행

pytorch가 순전파를 실행하는 동안 연산 기록을 저장하고 `backward()`가 그래프를 출력에서 입력 방향으로 거슬러 올라가며 기울기를 구함

7. 심층 신경망

7.1 심층 신경망이란?

- 은닉층이 2개 이상인 신경망
- 층이 깊어질수록 단순한 특징에서 복잡한 특징으로 계층적으로 학습함
- 다층 퍼셉트론과 구조는 같으며, 은닉층이 많은 경우를 **DNN**이라고 부름
- 기본적으로 위에서 설명했던 구조를 근원으로 파생됨

7.2 문제점

- 추가된 계층들이 학습 데이터의 낮은 의존성을 가지는 모형화가 가능하도록해 주기 때문에
과적합과 높은 시간 복잡도가 흔히 발생함
- 학습 시킬 때 시간 복잡도가 매우 높으므로 크기, 학습률, 초기 가중치 등 많은 매개변수를 고려해야한다.

7.3 주요 활용 분야

- 이미지 및 비디오 인식: 사물 찾는 객체 인식, 사람 얼굴 구별, **CT나 MRI**같은 의료 영상 분석
- 음성 및 언어 처리: 사람의 목소리 -> 글로 변환
- 금융 및 보안: 신용카드 결제 중 이상 거래를 잡아내는 사기 탐지 시스템에 활용됨

7.4 pytorch로 구현하기

1) 대응 관계

개념	pytorch
신경망 전체	nn.Module을 상속한 클래스
층 정의	__init__의 nn.Linear, 활성화 함수
순전파	forward(self, x)
역전파	직접 작성 안함, loss.backward()가 자동으로 처리
입력 펼치기	x.view(x.size(0), -1) 또는 nn.Flatten()
파라미터 수	sum(p.numel() for p in model.parameters())

2) 코드 구현

```
class DNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 100),
            nn.ReLU(),
            nn.Linear(100, 10)
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.net(x)

model = DNN()
print(sum(p.numel() for p in model.parameters()))
```

6.1에서 예시 들었던 값과 동일하게 구현함.

super().__init__()은 nn.Module의 초기화를 실행하는 부분으로
이 호출이 있어야 nn.Linear 등의 파라미터로 자동 등록됨

forward에는 순전파 흐름만 쓰고 역전파 코드는 따로 작성하지않음
nn.Module이 연산 기록을 남기기 때문에 자동으로 계산됨

은닉층을 추가하려면 nn.Linear와 nn.ReLU를 Sequential에
이어붙이기만하면 됨

8. 추가 조사

8.1 SoftMax와 CrossEntropyLoss

1) SoftMax란?

- 신경망의 출력값(점수) 여러 개를 모두 더하면 1이 되는 확률 분포로 바꿔 주는 활성화 함수
- 주로 여러 클래스 중 하나를 고르는 다중 분류 문제의 출력층에서 사용함
- 마지막 `nn.Linear`가 내보내는 아직 확률이 아닌 점수를 **logit**이라고 하며, **Softmax**가 이 **logit**을 확률로 변환

2) 수식

- $p_i = \exp(z_i) / \sum \exp(z_j)$ (z_i : i 번째 클래스의 **logit**, j 는 모든 클래스에 대해 더함)
- 분자: 각 **logit**에 지수함수를 적용함
- 분모: 모든 클래스의 **exp** 값의 합으로 나눠서 정규화함
- 그 결과 모든 p_i 는 0보다 크고 1보다 작으며, $\sum(p_i) = 1$ 이 됨

3) 왜 필요한가?

- 4장의 **MSE**는 회귀 문제에 적합
- 그러나 공/골대/로봇 처럼 여러 클래스 중 하나를 고르는 분류의 문제는 출력이 “각 클래스일 확률” 이어야함.
- 3.2 에서 시그모이드가 0~1 사이 값을 출력할 수 있기 때문에 확률로 표현 가능하다고 했는데 클래스가 3개 이상이면 시그모이드 만으로는 확률의 합이 1이 되지않아서 **SoftMax**를 사용함
- **logit**이 크면 **exp**가 매우 커져서 오버플로우가 발생할 수 있는데 **SoftMax**는 모든 **logit**에서 같은 값을 빼도 결과가 변하지않는 성질이 있음